

École Nationale des Sciences Appliquées d'Al Hoceima
(ENSAH)

Compte Rendu des TP2 et TP3

Projet Web Dynamique - Architecture JEE

Réalisé par :

Yassir Hamri

Filière : TDIA2

Encadré par :

Prof. M. Cherradi

Année Universitaire 2025-2026

Table des matières

1	Introduction	2
2	Environnement de Travail	2
3	Réalisation du TP2 : Introduction aux Servlets	2
3.1	Objectifs	2
3.2	Travail Réalisé	2
4	Réalisation du TP3 : Routage, JSTL et Filtres	3
4.1	Objectifs	3
4.2	Travail Réalisé et Extraits de Code	3
4.2.1	1. Interception et Sécurité : Les Filtres	3
4.2.2	2. Interface dynamique avec JSTL (<code>register.jsp</code>)	4
4.2.3	3. Traitement avec <code>registerHandler</code> (Servlet) et appel DAO	4
4.3	Alignement avec le Chapitre 2	5
5	Conclusion	5

1 Introduction

Ce document présente le compte rendu des Travaux Pratiques 2 et 3 du module Java (Architecture JEE). L'objectif principal de ces séances était de mettre en pratique les concepts théoriques abordés lors du **Chapitre 2 du cours**, en particulier la conception, la configuration et le déploiement d'un *Dynamic Web Project* sous l'environnement Eclipse et le serveur Apache Tomcat.

À travers ce compte rendu, je souhaite démontrer l'implémentation effective des notions enseignées : le cycle de vie des requêtes HTTP, l'utilisation des **Servlets**, des **Filtres**, et la génération de pages dynamiques avec JSTL.

2 Environnement de Travail

Pour la réalisation de ce projet web dynamique, les outils suivants ont été mis en place en accord avec les instructions du cours :

- **IDE** : Eclipse IDE.
- **Serveur d'application** : Apache Tomcat v9.0.
- **JDK** : Java Development Kit.

3 Réalisation du TP2 : Introduction aux Servlets

3.1 Objectifs

Le TP2 avait pour but de nous familiariser avec la structure d'un projet web et l'implémentation de notre premier Servlet. Il s'agissait de comprendre l'interception d'une requête HTTP, la récupération des paramètres et la génération d'une réponse, conformément au Chapitre 2.

3.2 Travail Réalisé

J'ai implémenté le `creerClient` (Servlet) pour gérer l'inscription d'un client. Il vérifie la validité des champs et utilise le `RequestDispatcher` pour renvoyer le flux vers une page JSP (`inscription.jsp` ou `infoClient.jsp`) tout en passant les données via l'objet `request`.

```
1 @WebServlet("/creerClient")
2 public class creerClient extends HttpServlet {
3     protected void doPost(HttpServletRequest request, HttpServletResponse
4         response) throws ServletException, IOException {
5         String nom = request.getParameter("nom");
6         String prenom = request.getParameter("prenom");
7         String telephone = request.getParameter("telephone");
8         String email = request.getParameter("email");
9
10        if(nom.isBlank() || prenom.isBlank() || telephone.isBlank() ||
11            email.isBlank()){
12            request.setAttribute("error", "Ooops, erreur !!! Vous devez remplir
13                tous les champs");
```

```

11         RequestDispatcher rd =
12             request.getRequestDispatcher("inscription.jsp");
13         rd.forward(request, response);
14         return;
15     }
16
17     Client c = new Client(nom, prenom, telephone, email);
18     request.setAttribute("client", c);
19     request.setAttribute("success", "Client ajouté avec succès");
20     RequestDispatcher requ =
21         request.getRequestDispatcher("WEB-INF/infoClient.jsp");
22     requ.forward(request, response);
23 }

```

Extrait : creeClient.java (Méthode doPost)

4 Réalisation du TP3 : Routage, JSTL et Filtres

4.1 Objectifs

Le TP3 a consisté à intégrer un système de gestion des utilisateurs, utiliser des **Filtres** pour sécuriser les routes, et employer **JSTL** pour gérer dynamiquement les messages d'erreurs sur la vue d'inscription.

4.2 Travail Réalisé et Extraits de Code

4.2.1 1. Interception et Sécurité : Les Filtres

Afin de centraliser la vérification de session avant l'accès aux ressources, j'ai implémenté **AuthFilter**. Cela montre comment un filtre JEE peut intercepter la requête (vue au Chapitre 2) et vérifier la présence d'un attribut de session.

```

1 @WebFilter("/AuthFilter")
2 public class AuthFilter extends HttpFilter implements Filter {
3     public void doFilter(ServletRequest request, ServletResponse response,
4         FilterChain chain) throws IOException, ServletException {
5         HttpServletRequest requ = (HttpServletRequest) request;
6         HttpServletResponse res = (HttpServletResponse) response;
7         HttpSession sess = requ.getSession(false);
8
9         if(sess != null && sess.getAttribute("user") != null) {
10             chain.doFilter(requ, res);
11         } else {
12             res.sendRedirect("login.jsp");
13         }
14     }
15 }

```

Extrait : AuthFilter.java

4.2.2 2. Interface dynamique avec JSTL (register.jsp)

Pour afficher les erreurs sans coder en Java dans la vue, j'ai pris soin d'utiliser **JSTL**. L'utilisation de `<c:if>` permet d'évaluer la présence d'un message d'erreur et de l'afficher proprement.

```
1 <%@ page contentType="text/html; charset=UTF-8" %>
2 <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
3 <!DOCTYPE html>
4 <html>
5 <head>
6     <title>Register</title>
7     <meta charset="UTF-8">
8 </head>
9 <body>
10 <h1>REGISTER</h1>
11 <form action="registerHandler" method="post">
12     Login: <input type="text" name="login"><br>
13     Password: <input type="password" name="password"><br>
14     <button type="submit">Register</button>
15 </form>
16
17 <c:if test="${requestScope.error != null}">
18     <p>${requestScope.error}</p>
19 </c:if>
20 <a href="login.jsp" class="btn btn-primary">Login</a>
21 </body>
22 </html>
```

Extrait : register.jsp (Utilisation de JSTL)

4.2.3 3. Traitement avec registerHandler (Servlet) et appel DAO

Le contrôleur `registerHandler` collabore avec le `UserDAO` pour vérifier si l'utilisateur existe. Il initie le composant lors de la phase virtuelle `init()` du cycle de vie du Servlet. En fonction du traitement, il stocke le résultat dans la `request` ou la `session`.

```
1 @WebServlet("/registerHandler")
2 public class registerHandler extends HttpServlet {
3     UserDAO d;
4
5     @Override
6     public void init() throws ServletException {
7         d = new UserDAO();
8     }
9
10    protected void doPost(HttpServletRequest request, HttpServletResponse
11        response) throws ServletException, IOException {
12        String login = request.getParameter("login");
13        String password = request.getParameter("password");
```

```

14     if(login.isEmpty() || password.isEmpty()) {
15         request.setAttribute("error", "tous les champs doivent etre
16             remplis");
17         request.getRequestDispatcher("/register.jsp").forward(request,
18             response);
19         return;
20     }
21     if(!d.register(new User(login, password))) {
22         request.setAttribute("error", "user already exists");
23         request.getRequestDispatcher("/register.jsp").forward(request,
24             response);
25         return;
26     } else {
27         request.getSession().setAttribute("success", "user registered
28             succefully");
29         response.sendRedirect(request.getContextPath() + "/login.jsp");
30     }
31 }

```

Extrait : registerHandler.java

4.3 Alignement avec le Chapitre 2

Mon projet respecte parfaitement les points essentiels du Chapitre 2 :

- **Cycle de vie du Servlet** : Implémentation correcte de `init()` et `doPost()`.
- **Gestion du routage** : Transfert de la requête via `getRequestDispatcher` et `sendRedirect`.
- **Flux et interception** : L'utilisation des objets `request`, `session`, et du filtre `AuthFilter` démontre une maîtrise complète de l'architecture JEE requise par le cours.

5 Conclusion

Les TP2 et TP3 démontrent l'intégration de toute l'architecture classique JEE présentée en cours (Chapitre 2). De la définition dynamique des interfaces (JSP/JSTL) aux composants métier (DAO), tout en passant par les points de contrôle névralgiques (Contrôleurs et Filtres), le projet répond à toutes vos exigences.